# Acta de Constitución del Proyecto — TrazaMed

Plataforma web de gestión de inventario, trazabilidad por lotes y pedidos para distribución de insumos médicos

**Versión:** 0.1 (borrador) · **Fecha:** 31/10/2026 · **Autor:** Bryan Gonzales · **Estado:** Pendiente de aprobación

## 1. Información general del proyecto

- **Nombre del proyecto:** TrazaMed
- **Código:** TM-001
- **Organización simulada (cliente):** MediDistrib S.A.
- **Patrocinador (rol simulado):** Gerencia General de la organización simulada
- **Gerente de proyecto / Product Owner / Líder técnico:** Bryan Gonzales
- **Fecha de inicio:** 02/11/2026
- **Fecha de finalización estimada:** 05/02/2027 (14 semanas, 7 sprints de 2 semanas)
- **Colchón de contingencia:** hasta 19/02/2027 (2 semanas, considerando festividades y carga académica)
- **Dedicación:** mínimo 3 horas diarias (≈ 20 horas semanales, ≈ 280 horas totales)
- **Repositorio:** https://github.com/BryanGonz180/TrazaMed
- **Tablero de gestión:** https://github.com/BryanGonz180/TrazaMed/projects

## 2. Propósito y justificación

### 2.1 Problema

Las distribuidoras de insumos médicos manejan productos con **lote, fecha de caducidad, registro sanitario y ubicación física**. Cuando este control se lleva en hojas de cálculo o de forma manual aparecen problemas recurrentes:

- Pérdidas por productos que vencen en bodega sin que nadie lo advierta a tiempo.
- Despachos de lotes incorrectos (se entrega un lote más nuevo mientras otro está por vencer).
- Falta de trazabilidad: ante un reclamo de un hospital o clínica no se puede reconstruir qué lote se entregó, cuándo y quién lo autorizó.
- Inconsistencias de stock por errores de digitación y por dos personas reservando el mismo producto.
- Reportes lentos y poco confiables para la gerencia.

## 2.2 Oportunidad

Desarrollar un sistema web que automatice el control por lote, asigne el despacho con la regla FEFO (primero en vencer, primero en salir), genere alertas preventivas y deje un registro auditable de cada movimiento.

## 2.3 Propósito formativo

El proyecto cumple una doble función: resolver un problema real de un sector regulado y servir como **experiencia práctica de ingeniería de software de nivel profesional** (metodología ágil, calidad, seguridad, CI/CD y documentación) que se presentará como portafolio profesional.

# 3. Objetivos

## 3.1 Objetivo general

Diseñar, construir y desplegar una aplicación web de gestión de inventario y trazabilidad para una distribuidora de insumos médicos, aplicando prácticas de desarrollo de software vigentes en la industria, antes del 05/02/2027.

## 3.2 Objetivos específicos (SMART)

- **OE-1:** Implementar el control de inventario por producto, lote, fecha de caducidad y ubicación de bodega, con kardex completo de movimientos, al finalizar el Sprint 3.
- **OE-2:** Implementar la asignación automática de lotes con regla FEFO en pedidos y despachos, con pruebas automatizadas que cubran los casos límite, al finalizar el Sprint 4.
- **OE-3:** Garantizar la consistencia del stock ante operaciones simultáneas (sin sobreventa en pruebas de concurrencia), al finalizar el Sprint 4.
- **OE-4:** Automatizar alertas de vencimiento próximo y stock mínimo, y la generación de reportes programados (PDF/Excel), al finalizar el Sprint 5.
- **OE-5:** Alcanzar una cobertura de pruebas automatizadas de al menos 80% en la lógica de negocio central y un pipeline de integración continua que ejecute las pruebas en cada Pull Request.
- **OE-6:** Desplegar la aplicación en un entorno accesible por internet con datos simulados y documentación completa antes de la fecha de cierre.

# 4. Alcance

## 4.1 Dentro del alcance (MVP)

- Autenticación y autorización por roles (Administrador, Bodega, Ventas, Gerencia).
- Catálogo de productos, categorías, proveedores y clientes (hospitales, clínicas, farmacias).

- Gestión de lotes con fecha de caducidad y registro sanitario.
- Ubicaciones de bodega (bodega, estante, caja) y movimientos de inventario (entradas, salidas, ajustes, transferencias).
- Órdenes de compra a proveedores y recepción de mercadería.
- Cotizaciones y pedidos de clientes con reserva de stock y despacho FEFO.
- Kardex y registro de auditoría (quién, qué, cuándo).
- Alertas de vencimiento y stock mínimo; reportes en PDF y Excel.
- Dashboard gerencial con indicadores clave.
- API REST documentada y aplicación web (frontend).

## 4.2 Fuera del alcance (primera versión)

- Facturación electrónica ante el SRI y contabilidad.
- Integración con sistemas reales de la empresa donde Bryan trabaja.
- Aplicación móvil nativa.
- Pasarelas de pago.
- Uso de datos reales de clientes, productos o precios de cualquier empresa.

## 4.3 Supuestos

- Se dispondrá de al menos 3 horas diarias de trabajo efectivo.
- Las herramientas elegidas tienen versión gratuita o de estudiante suficiente para el proyecto.
- Los datos serán totalmente ficticios y generados para el proyecto.
- Las reglas de negocio se levantarán a partir del conocimiento general del sector, sin copiar información confidencial.

## 4.4 Restricciones

- Equipo de una sola persona que rota roles (Product Owner, arquitecto, desarrollador, QA, DevOps y redactor técnico).
- Presupuesto económico objetivo: 0 USD (capas gratuitas).
- Compatibilidad con las actividades académicas y laborales del autor.
- Stack principal basado en Java y PostgreSQL.

## 5. Entregables principales

- **E-1:** Acta de proyecto (este documento) y plan de trabajo (backlog y roadmap).
- **E-2:** Documento de requisitos (historias de usuario con criterios de aceptación).
- **E-3:** Diseño: modelo entidad-relación, diagramas de arquitectura (modelo C4) y registros de decisiones de arquitectura (ADR).
- **E-4:** Código fuente en GitHub con historial limpio, pruebas y pipeline de CI.
- **E-5:** API REST documentada (OpenAPI/Swagger) y aplicación web funcional.
- **E-6:** Aplicación desplegada con datos simulados y contenedores Docker reproducibles.

- **E-7:** Documentación: README profesional, manual de usuario, guía de instalación, plan de pruebas y registro de cambios.
- **E-8:** Material de portafolio: video demostrativo, capturas y caso de estudio.

## 6. Requisitos de alto nivel

### 6.1 Requisitos funcionales

- **RF-01:** El sistema permitirá iniciar sesión y restringirá cada función según el rol del usuario.
- **RF-02:** El sistema permitirá registrar productos, proveedores y clientes.
- **RF-03:** El sistema registrará lotes con número de lote, fecha de caducidad, registro sanitario y cantidad.
- **RF-04:** El sistema registrará todo movimiento de inventario en un kardex inmutable.
- **RF-05:** El sistema asignará automáticamente los lotes a un pedido según la regla FEFO.
- **RF-06:** El sistema impedirá reservar o despachar más stock del disponible, incluso con usuarios simultáneos.
- **RF-07:** El sistema generará alertas de lotes próximos a vencer y de productos bajo el stock mínimo.
- **RF-08:** El sistema permitirá gestionar órdenes de compra, recepción, cotizaciones y pedidos.
- **RF-09:** El sistema generará reportes exportables (PDF y Excel) y un dashboard gerencial.
- **RF-10:** El sistema registrará una auditoría de acciones de usuarios.

### 6.2 Requisitos no funcionales (modelo de calidad ISO/IEC 25010)

- **Seguridad:** autenticación con tokens, contraseñas con hash seguro, validación de entradas y controles alineados al OWASP Top 10 y OWASP ASVS (nivel 1 como mínimo).
- **Fiabilidad:** transacciones ACID en operaciones de inventario; respaldos de base de datos documentados.
- **Rendimiento:** respuestas de la API en menos de 500 ms en el percentil 95 con el volumen de datos de prueba definido (≥ 10.000 movimientos).
- **Mantenibilidad:** arquitectura por capas, código revisado en Pull Requests, análisis estático y cobertura ≥ 80% en la lógica central.
- **Usabilidad:** interfaz responsiva y flujos principales completables en pocos pasos.
- **Portabilidad:** ejecución reproducible con Docker y configuración por variables de entorno (principios de la aplicación de 12 factores).
- **Observabilidad:** registros estructurados (logs) y endpoint de salud.

## 7. Estándares y buenas prácticas adoptadas

- **Gestión del proyecto:** Scrum (Guía Scrum 2020) y enfoque de gestión alineado al PMBOK 7.

- **Requisitos:** buenas prácticas de ISO/IEC/IEEE 29148 (requisitos identificados, trazables y verificables) e historias de usuario con criterios de aceptación.
- **Calidad de producto:** ISO/IEC 25010.
- **Seguridad:** OWASP Top 10, OWASP ASVS y revisión periódica de dependencias vulnerables.
- **Arquitectura y diseño:** modelo C4, ADR, principios SOLID y arquitectura por capas.
- **Control de versiones:** GitHub Flow (ramas cortas y Pull Requests), Conventional Commits y versionado semántico (SemVer), con protección de la rama principal.
- **Entrega continua:** integración continua con GitHub Actions; métricas DORA como referencia de mejora.
- **API:** diseño REST y contrato OpenAPI 3.
- **Documentación:** README, changelog (Keep a Changelog), runbook operativo y manual de usuario.
- **Datos personales:** principios de la Ley Orgánica de Protección de Datos Personales del Ecuador. Como todo dato es ficticio, el proyecto se diseña igualmente aplicando minimización y control de acceso.
- **Sector salud (referencia funcional):** los conceptos de lote, caducidad y registro sanitario se modelan según la lógica de la normativa ARCSA para dispositivos médicos. [COMPLETAR: verificar y citar la regulación vigente aplicable al redactar los requisitos]

## 8. Arquitectura y tecnologías

- **Backend:** Java 21, Spring Boot (Web, Data JPA, Security, Validation).
- **Base de datos:** PostgreSQL administrado con pgAdmin; migraciones versionadas con Flyway.
- **Frontend:** React con Vite. [COMPLETAR: confirmar o cambiar por Thymeleaf + Bootstrap si se prefiere una curva más suave]
- **Calidad:** JUnit 5, Mockito, Testcontainers, JaCoCo (cobertura) y análisis estático.
- **DevOps:** Git, GitHub, GitHub Actions, Docker y Docker Compose.
- **Documentación técnica:** Swagger/OpenAPI, diagramas en Mermaid o draw.io.
- **Despliegue:** plataforma en la nube con capa gratuita vigente [COMPLETAR: elegir y verificar condiciones al llegar al Sprint 7].
- **Estilo arquitectónico:** monolito modular por capas (controlador, servicio, dominio, repositorio), diseñado para evolucionar a servicios si fuera necesario.

## 9. Metodología y gobierno del proyecto

- **Marco de trabajo:** Scrum con sprints de 2 semanas.
- **Ceremonias (adaptadas a un equipo de una persona):** planificación de sprint, revisión diaria breve de avance por escrito, revisión del sprint con demo grabada y retrospectiva con acciones de mejora.

- **Artefactos:** backlog priorizado, objetivo de sprint, incremento funcional y gráfico de avance.
- **Definición de Listo (DoR):** historia con descripción, criterios de aceptación, estimación y dependencias identificadas.
- **Definición de Terminado (DoD):** código integrado mediante Pull Request, pruebas automatizadas aprobadas, sin vulnerabilidades críticas conocidas, documentación actualizada, criterios de aceptación verificados y entrega desplegable.
- **Flujo de Git:** rama principal protegida; ramas por funcionalidad (feature/…); commits con Conventional Commits; Pull Requests con plantilla y lista de verificación, incluso trabajando solo.
- **Roles simulados:** Product Owner, Arquitecto, Desarrollador, QA, DevOps y Redactor técnico (todos asumidos por Bryan, registrando en cada tarea con qué rol se trabajó).
- **Gestión de cambios:** todo cambio de alcance se registra en la sección 18 y se prioriza en el backlog.

## 10. Cronograma de alto nivel

- **Sprint 0 (02/11 – 13/11/2026), Descubrimiento y preparación:** acta aprobada, requisitos, backlog, modelo de datos, repositorio y entorno de desarrollo.
- **Sprint 1 (16/11 – 27/11/2026), Base técnica:** autenticación y roles, estructura de la API, migraciones y CI inicial.
- **Sprint 2 (30/11 – 11/12/2026), Catálogo y proveedores:** productos, lotes y registro sanitario con validaciones y pruebas.
- **Sprint 3 (14/12 – 25/12/2026), Inventario y kardex:** movimientos, ubicaciones de bodega y auditoría.
- **Sprint 4 (28/12/2026 – 08/01/2027), Pedidos y FEFO:** cotizaciones, reservas, despacho y control de concurrencia.
- **Sprint 5 (11/01 – 22/01/2027), Automatización y reportes:** alertas, tareas programadas y reportes PDF/Excel.
- **Sprint 6 (25/01 – 05/02/2027), Dashboard y frontend final:** panel gerencial y pulido de interfaz.
- **Sprint 7 (08/02 – 19/02/2027), Producción y cierre:** despliegue, monitoreo, manual de usuario, video demo y caso de estudio.

Nota: el cronograma consume el colchón de contingencia; si hay retrasos se reduce el alcance del backlog, no la calidad.

## 11. Criterios de éxito e indicadores

- Todos los requisitos RF-01 a RF-10 verificados con sus criterios de aceptación.
- Cobertura de pruebas ≥ 80% en la lógica de negocio central.
- Cero vulnerabilidades críticas abiertas en dependencias al momento de la entrega.
- Prueba de concurrencia aprobada: sin sobreventa con operaciones simultáneas.
- Pipeline de CI en verde en la rama principal.
- Aplicación desplegada, accesible y con datos de ejemplo.
- Documentación completa y repositorio presentable como portafolio.
- Indicadores de seguimiento: velocidad por sprint, historias completadas frente a planificadas, tiempo de ciclo de los Pull Requests y tasa de éxito del pipeline.

## 12. Riesgos iniciales

- **R-01, Sobrecarga académica o laboral reduce las horas disponibles.** Probabilidad: alta. Impacto: alto. Respuesta: calendario semanal fijo, colchón de 2 semanas y backlog priorizado para recortar alcance.
- **R-02, Curva de aprendizaje de Spring Boot y React.** Probabilidad: alta. Impacto: medio. Respuesta: sprint inicial con pruebas de concepto y tiempo de estudio explícito.
- **R-03, Crecimiento descontrolado del alcance.** Probabilidad: media. Impacto: alto. Respuesta: MVP definido, lista de funcionalidades fuera de alcance y control de cambios.
- **R-04, Complejidad de la concurrencia y la lógica FEFO.** Probabilidad: media. Impacto: alto. Respuesta: diseñar y probar primero el núcleo de reserva y despacho, con pruebas de concurrencia tempranas.
- **R-05, Pérdida de código o de documentación.** Probabilidad: baja. Impacto: alto. Respuesta: commits frecuentes a GitHub y respaldos de la base de datos.
- **R-06, Uso accidental de información confidencial de la empresa real.** Probabilidad: media. Impacto: alto. Respuesta: solo datos ficticios, revisión del repositorio antes de cada publicación y confirmación con el empleador si se inspira en procesos reales.
- **R-07, Cambios en planes gratuitos de las herramientas en la nube.** Probabilidad: media. Impacto: bajo. Respuesta: Docker para portabilidad y alternativas identificadas.
- **R-08, Pérdida de motivación o abandono.** Probabilidad: media. Impacto: alto. Respuesta: demos cortas cada sprint, avance visible en GitHub y metas semanales pequeñas.

## 13. Interesados y roles

- **Patrocinador (simulado):** Gerencia General; aprueba el alcance y recibe las demos.
- **Product Owner:** Bryan; define y prioriza el backlog.
- **Equipo de desarrollo:** Bryan en sus roles de arquitecto, desarrollador, QA y DevOps.
- **Usuarios clave (simulados):** personal de bodega, ventas, compras y gerencia.

- **Revisor externo (recomendado):** un docente, compañero o profesional [COMPLETAR: nombre] que revise el código y la documentación al cierre de algunos sprints para simular revisión por pares.

## 14. Recursos y presupuesto

- **Tiempo:** ≈ 280 horas (3 horas diarias × 14 semanas × ~6,7 días efectivos por semana, con ajuste por imprevistos).
- **Hardware y software:** equipo personal del autor; VS Code, Java JDK, PostgreSQL, pgAdmin, Git, Git Bash, Docker y Node.js.
- **Servicios:** GitHub (gratuito), plataforma de despliegue en capa gratuita.
- **Costo económico estimado:** 0 USD.

## 15. Seguridad, privacidad y cumplimiento

- Todos los datos del proyecto serán ficticios o generados sintéticamente.
- No se publicará en el repositorio ninguna credencial, clave o dato real (uso de variables de entorno y archivo .gitignore).
- Se mantendrá un análisis de dependencias vulnerables y revisión de seguridad básica antes de cada versión.
- Si se toman ideas del entorno laboral del autor, se tratarán como conocimiento general del sector, sin reproducir procedimientos, precios, nombres de clientes ni datos internos de ninguna empresa.

## 16. Estrategia de documentación y portafolio

- **README profesional:** problema, solución, capturas, arquitectura, instrucciones de ejecución y tecnologías.
- **Carpeta docs/ en el repositorio:** acta, requisitos, ADR, diagramas, plan de pruebas, manual de usuario y runbook.
- **Historial de Git:** commits claros y Pull Requests descriptivos como evidencia de proceso.
- **Registro de cambios:** changelog por versión.
- **Evidencias por sprint:** demo breve grabada y resumen de retrospectiva.
- **Material final:** video de 2 a 3 minutos, caso de estudio para LinkedIn y enlace al despliegue.

## 17. Aprobaciones

- **Patrocinador (simulado)**
- **Gerente de proyecto / Product Owner:** Bryan Gonzales

## 18. Control de cambios

- **v0.1 — 31/10/2026 — Bryan:** versión inicial del acta (borrador).
- [COMPLETAR: agregar una línea por cada cambio con versión, fecha, autor y descripción]

## Anexo A. Plan de la primera semana (Sprint 0)

- **Día 1:** crear el repositorio en GitHub (README inicial, .gitignore, licencia, estructura de carpetas) y el tablero de proyecto con columnas Backlog, Listo, En curso, En revisión y Hecho.
- **Día 2:** completar los campos [COMPLETAR] de esta acta y fijar el nombre de la organización simulada.
- **Día 3:** levantar el proceso actual (AS-IS) del control de inventario, vencimientos y pedidos; dibujar el flujo en un diagrama.
- **Día 4:** listar los problemas y necesidades por rol (bodega, ventas, compras y gerencia).
- **Día 5:** redactar las primeras 15 historias de usuario con criterios de aceptación y priorizarlas.
- **Días 6 y 7:** diseñar el primer borrador del modelo entidad-relación en PostgreSQL y preparar el entorno con Spring Boot y Docker Compose.

## Anexo B. Plantilla de historia de usuario

- **ID:** HU-000
- **Como** [rol], **quiero** [funcionalidad], **para** [beneficio].
- **Criterios de aceptación:** dado [contexto], cuando [acción], entonces [resultado esperado].
- **Prioridad:** Alta, Media o Baja.
- **Estimación:** [puntos o horas].
- **Dependencias:** [HU relacionadas].
- **Notas técnicas:** [decisiones, riesgos, enlaces].